GPU Algorithms for Biconnected Components on Large Graphs

Abhijeet Sahu*, Andaluri S P V M Aditya*, G Ramakrishna*, Kishore Kothapalli†, and Dip Sankar Banerjee‡
*IIT Tirupati, India, {cs22s501, cs21b002, rama}@iittp.ac.in
†IIIT Hyderabad, India, kkishore@iiith.ac.in
‡IIT Jodhpur, India, dipsankarb@iitj.ac.in

***Abstract*—Efficient algorithms for graph biconnectivity are vital in diverse domains, ranging from medical diagnostics to chip design. Recently, there has been a surge of accelerator based algorithms to tackle the scale challenge and benefit from parallelism. However, a key limitation is the available device memory which renders existing approaches unusable. We refer this to as the Out-of-Memory (OOM) setting. This constraint demands a fundamental rethinking of both algorithmic design and implementation strategies. In this paper, we address this challenge by introducing a novel parallel edge-pruning technique that reduces the memory footprint required for biconnected component decomposition.**

Crucially, unlike prior methods that rely on a computationally expensive BFS tree, we prove that specific non-tree edges with respect to an arbitrary spanning tree can be safely pruned without altering biconnectivity. This distinction allows us to construct an efficient $O(n)$-size skeleton without incurring the overheads of BFS. Based on this theory, we present two GPU-based parallel algorithms: FAST-BCC-FILTER for in-memory BCC computation and its out-of-core extension, PEA-BCC.

We evaluate our approach on 30 real-world datasets. For massive graphs resulting in the OOM setting, our approach outperforms the best CPU-based baseline by $2.7\times$ on average. Furthermore, for graphs that do not result in the OOM setting, we achieve an average speedup of $71\times$ over the state-of-the-art GPU baseline.

***Keywords*-Large-scale graphs, biconnected components, parallel algorithms, graph algorithms**

1. Introduction

The efficient processing of large graphs is increasingly critical across various computational domains, as recent studies ([24], [25]) highlight the growing complexity and scale of networked data. Over the past decade, substantial research has been conducted on graph analytics using parallel architectures and accelerators, including multi-core CPUs and GPUs. This research has resulted in an extensive body of literature that includes parallel algorithms, their implementations, libraries, Domain Specific Languages, benchmark datasets, and programming frameworks. However, a significant challenge that remains is effectively handling the ever-growing size of real-world graph datasets.

Graph datasets are expanding rapidly, with graphs containing billions of nodes and edges becoming commonplace. For example, the current graph representing Reddit includes over 500 million vertices and 137 billion edges, requiring more than 500 GB of storage space. While modern server-class CPUs have RAM capacities in the range of hundreds of gigabytes, the memory on accelerators, such as GPUs, is limited to tens of gigabytes*. Despite these memory constraints, GPUs have emerged as a powerful solution for general-purpose computation over the last two decades thanks to their massive parallelism and high memory bandwidth. Due to this, a large body of research exists on how to efficiently use GPUs for irregular problems, such as graph workloads, by addressing impediments including load imbalance, memory access latency, thread synchronization, and others.

GPU-based graph algorithms for a variety of problems including BFS, shortest paths, centrality metrics, k-connectivity for $k \geq 1$, offer impressive speedup over shared-memory counterparts. Yet, as the size and complexity of graph-structured data continue to grow, the limited on-board memory of GPUs presents new hurdles. Thus, addressing these constraints through innovative out-of-memory strategies has become imperative.

In this paper, we focus on the graph biconnectivity problem, a fundamental primitive in graph theory that has been studied extensively for decades. The *biconnected components* are maximal subgraphs of a graph that remain connected even after the removal of a single vertex. Graph biconnectivity finds applications in multiple problems ranging from planarity testing [11], VLSI design [21] to social network analysis [6].

Sequential algorithms for the graph biconnectivity problem such as the algorithm by Hopcroft and Tarjan [14] or the more recent algorithm by Schmidt [26] use the depth-first traversal of a graph. These algorithms are not parallelizable owing to DFS being P-complete [23]. In the Parallel Random Access Machine (PRAM) model [15], Tarjan and Vishkin proposed the first PRAM algorithm [30]. Since then, much effort has focused on multi-core and many-core architectures [7], [9], [10], [31]. Among recent advancements, **FAST-BCC** [10] stands out as a parallel multicore solution that improves upon the Tarjan–Vishkin (TV) algorithm by significantly reducing on space complexity. It achieves $O(n)$ extra space usage for intermediate data structures, marking a significant improvement in terms of space efficiency. However, despite these advances, a common assumption silently threads through nearly all these prior designs: **the entire graph and all intermediate data structures must fit in the memory of a single architecture**.

*For example, the latest NVIDIA L40s GPU offers 48 GB of GDDR6 DRAM.

As the scale of modern graphs continue to grow into billions of edges, this assumption becomes increasingly untenable.

For simplicity, we say that the situation when the input and the intermediate data structures do not fit in the memory as **Out-of-Memory (OOM)**. In the OOM situation, assumptions to the contrary that existing algorithms make do not hold. This raises a critical question for algorithm designers: How to design an efficient algorithm under the OOM situation.

Formally, given an undirected, unweighted graph $G = (V, E)$ with n vertices and m edges, where the input and the data structures needed for execution exceeds available GPU memory, we aim to compute *cut vertices*, *cut edges*, and *biconnected component (*BCC*)* labels for all edges using an out-of-memory approach tailored for GPU execution.

BCC in the Out-of-Memory Setting

Input: An undirected graph $G = (V, E)$ such that $|E| = \mathcal{O}(m) \gg$ GPU memory capacity.

Output: Identify all *cut vertices*, *cut edges*, and *biconnected component (BCC) labels* in G.

Existing frameworks, such as HYPAR [20], GRAPHCHI [17], and GRAPHREDUCE [27], alongside architectural features like UVM [12] and EMOGI [18], provide only partial solutions. Although existing algorithms can be refactored to fit these frameworks when data exceeds GPU memory, doing so restricts the design space and prevents the use of more efficient, custom-tailored algorithms suitable for OOM. Furthermore framework-based approaches suffer from limited expressibility; not *all* computations can be naturally mapped to the specific algorithmic constructs provided. Finally, approaches relying on architectural support like UVM or zero-copy mechanisms often yield suboptimal performance due to the irregular memory access patterns inherent to graph algorithms, which trigger frequent and costly data movement over the PCIe bus.

A. Motivation and Challenges

Algorithms that aim to reduce the memory footprint of biconnected component (BCC) computations often rely heavily on identifying and removing non-essential edges, a technique known as **edge pruning**. Cheriyan and Thurimella [8] were the first to observe that the vast majority of edges in a dense graph are non-essential for determining biconnectivity. They show that biconnectivity can be preserved by a sparse subgraph, or *certificate*, containing only $O(n)$ edges. In the context of parallel computing, Cong and Bader [9] successfully applied this theory to optimize the memory usage of the TV algorithm [30]. By preprocessing the input graph to filter redundant edges, they significantly reduce the space complexity of the algorithm's auxiliary structures. Their method defines the sparse certificate H, referred to as the skeleton of G, as the union of a BFS tree T and a spanning forest F of the remainder graph $G - T$. This resulting subgraph $H := T \cup F$ contains at most $2n - 2$ edges. Cong and Bader proved that the original graph G is biconnected if and only if this skeleton H is biconnected.

However, a major drawback of this approach is its reliance on a BFS spanning tree. The pruning logic cannot correctly filter cross-edges if the underlying tree is an arbitrary spanning tree. This distinction is critical in our context because constructing a strict BFS tree on massive graphs, especially in **Out-of-Memory** (OOM) setting is highly non-trivial and computationally expensive.

Table I summarizes the major approaches, highlighting the trade-offs between skeleton size and algorithmic depth. We use the standard PRAM model terminology [15], where the depth of a parallel algorithm is defined as the length of the longest chain of dependencies. While traditional methods like Cong-Bader achieve a minimal skeleton size of $O(n)$, they incur high depth due to BFS construction. Conversely, approaches such as FAST-BCC [10] successfully reduce the size of intermediate data structures to $O(n)$, yet the overall memory footprint remains $O(m)$. We discuss these methods in greater detail in Section 1-C.

TABLE I: Skeleton size across various BCC algorithms. Here, k denotes the number of batches of edges in the input graph.

Algorithm	Skeleton Size	Depth
TV [30]	$\Theta(m)$	$\Theta(\log n)$
TV-filter [9]	$\Theta(n)$	$\Theta(dia.(G) + \log n)$
FAST-BCC [10]	$\Theta(m)$	$\Theta(\log n)$
Ours-FAST-BCC-Filter	$\Theta(n)$	$\Theta(\log n)$
Ours-PEA-BCC	$\Theta(n)$	$\Theta(k \cdot \log n)$

Challenges. Based on the above observations, designing an efficient parallel algorithm to solve the BCC problem on massive graphs requires addressing two major challenges highlighted in Table I:

1) **Sparse Skeleton Characterization:** How can we characterize a subgraph G' of G with size $O(n)$ such that the BCCs of G can be derived directly from the connected components (CC) of G'?
2) **Arbitrary Spanning Tree:** How can we design an algorithm that functions with *any* rooted spanning tree by eliminating the strict BFS requirement, thereby avoiding the $\Theta(diameter(G))$ depth of BFS on massive graphs?

B. Our Contributions

To address these challenges, this paper makes the following specific contributions:

- **Theoretical Foundation:** We design a novel edge-pruning technique that works on an *arbitrary* spanning tree (Theorem 3.3). This allows us to characterize an $O(n)$-size skeleton subgraph G' of G without relying on a BFS spanning tree. Crucially, we prove that the biconnected components (BCC) of G can be derived directly from the connected components (CC) of this sparse skeleton G'.
- **In-Memory Algorithm:** We present a GPU-based parallel algorithm, FAST-BCC-FILTER, designed for cases where

the input graph fits within GPU memory. The algorithm efficiently constructs the $O(n)$-size skeleton in parallel, bypassing the depth bottleneck of $\Theta(\text{diameter}(G))$ associated with BFS construction (Section 3).

- **OOM Algorithm:** We extend the filtering approach to the OOM setting by introducing PEA-BCC, a GPU-based algorithm capable of processing massive graphs that exceed available device memory (Section 4).
- **Experimental Evaluation:** We perform an extensive evaluation on 30 real-world datasets. In the in-memory settings, our algorithm achieves an average speedup of $71\times$ over the state-of-the-art (WK-17 [31] and WK-18 [16]) algorithms. Crucially, for the 8 very large datasets where prior GPU methods fail (OOM scenarios), our approach outperforms the best available CPU baseline (FAST-BCC [10]) by $2.7\times$ on average (Section 6).

To the best of our knowledge, ours is the first GPU algorithm (in both in-memory and out-of-memory settings) to compute biconnected components using an *arbitrary* spanning tree, completely eliminating the reliance on BFS traversal and its associated sequential bottlenecks.

C. Related Work

The classic sequential approach for graph biconnectivity is the Hopcroft–Tarjan algorithm [14], which uses depth-first search (DFS) and identifies BCCs and articulation points in linear time. Moving from sequential to parallel setting, Tarjan–Vishkin (TV) [30] proposed an alternative that replaces DFS with an *arbitrary spanning tree (*AST*)*, as DFS can be hard to parallise [23]. They characterize the skeleton for biconnectivity as an auxiliary graph G' such that the connected components of G' are the biconnected components of G. Despite its elegance and theoretical efficiency, one practical drawback is that $|V(G')| = \Theta(m)$, because each original edge becomes a vertex in G'. As some authors note [10], [9], this can lead to excessive memory usage and overhead.

Cong and Bader introduced *TV-filter* [9] that reduces the size of the auxiliary graph by removing *non-essential edges*. Specifically, they build a breadth-first search (BFS) tree T on the input graph G, along with a spanning forest F of $G - T$. Any edge not in T or F is classified as "non-essential" for biconnectivity.

Subsequent work by Ben-David et al. [3] took a different route, proposing a *write-efficient parallel algorithm* that labels the biconnected components in $O(\log^2 n)$ time with only $O(m)$ total writes. Their key insight introduced the notions of *critical edges* (later referred to as *fence edges* [10]) and *component heads*. By removing these critical edges and then applying a connectivity sub-routine on a reduced "skeleton" graph G', each vertex $v \in V(G)$ can be assigned a unique label according to the CC it belongs to. Rather than producing the standard BCC labeling array $K[\cdot]$ of size m, where $K[i]$ is the label of the biconnected component containing the i-th edge, they propose an alternative labeling that uses only $O(n)$ space. This representation effectively encodes the standard BCC output implicitly; one can retrieve the label of any edge (u, v) in $O(1)$ time by checking the shared label of u and v.

Later, Dong et al. [10] extended these ideas with a multicore implementation called *Fast*-BCC. They showed that both back edges and fence edges can be excluded when constructing the skeleton G', leaving only plain tree edges (non-fence) and cross edges in G'. Although theoretically sound, for massive graphs, the skeleton can still contain $\Omega(m)$ edges, making it impractical to store on resource-constrained devices such as GPUs.

At the other end of the design spectrum, a multitude of parallel BCC algorithms use a BFS-based spanning tree. Slota and Madduri [29] presented a multicore algorithm that relies on performing independent secondary BFS traversals. During the first BFS traversal, level information is obtained for all the vertices. In the second round, for each vertex u in G, a BFS traversal is initiated in parallel in $G - u$ and gets truncated if it reaches a vertex whose level is upper bounded by the level of the parent of u.

However, as observed by Wadekar and Kothapalli [31], adapting this strategy to the GPU architecture is non-trivial, as scheduling massive numbers of independent BFS traversals can be computationally infeasible on GPUs, causing significant thread divergence and load imbalance.

Addressing the lack of a viable GPU-native solution, Wadwekar and Kothapalli [31] introduced the first GPU-based BCC algorithm (WK-17). In their approach, a BFS tree T of an input graph G is constructed and then using fundamental cycles with respect to T, an auxiliary graph H is built. Most interestingly, the cut vertices of G are mapped to the cut edges of H. After removing the cut edges from H, each connected component in the resultant graph corresponds to a BCC in G.

In WK-18 [16], for a given graph G, a sparse spanning subgraph H of G is sampled, and the biconnected components (BCC) of G are found using WK-17 [31]. In the next step, a supergraph G' is created, where each vertex in G' corresponds to a BCC in H or a cut vertex in H. Later, the edges in $G - H$ are used to add appropriate edges in the supergraph G', in such a way that the supergraph acts as a certificate for the biconnectivity of G. In the last stage, BCCs are found in G' using WK-17 [31], and then use them to obtain BCCs of G.

2. Model and Preliminaries

In this section, we present the Parallel External Accelerator (PEA) model, which is designed to handle OOM situations. We then introduce the fundamental building blocks used to develop our BCC algorithms.

A. The Parallel External Accelerator (PEA) Model

The PEA model consists of a host CPU and an accelerator (e.g., a GPU) connected by a communication channel. The CPU memory, M_{CPU}, stores the complete input of size N; for a graph with n vertices and m edges, $N = \Theta(n + m)$. In contrast, the accelerator memory is limited to $M_{\text{GPU}} = \Theta(n)$. Since $M_{\text{CPU}} \gg M_{\text{GPU}}$, the model operates in an Out-of-Memory (OOM) setting in which the edge set cannot persist

on the accelerator, and algorithms may maintain only $O(n)$ total state, i.e., $O(1)$ state per vertex on average.

Consequently, algorithms in the PEA model must process the input in partitions. We define a *pass* as one complete transfer of the entire input graph to the accelerator memory. As passes occur sequentially, our primary objective is to minimize the total number of passes. Furthermore, to minimize I/O overhead, the algorithm maintains all partial outputs on the accelerator. Thus, the total device memory footprint comprises of resident input buffers, auxiliary data structures, and the output, and is strictly bounded by $O(n)$.

We remark that the PEA model we introduce in this paper is different from similar models in the extant literature [1], [2]. These models rely largely on shared memory parallelism having access to a large pool of global memory. The cores are loosely coupled and can support functional parallelism, whereas an architecture such as a GPU is best at data parallel algorithms. In addition, algorithms in prior models typically assume implicit access to the global memory directly, akin to the UVM model that Nvidia supports, whereas PEA enforces explicit memory management, requiring algorithms to manually stage data transfers between the host and the accelerator.

B. Building Blocks

We now review foundational subroutines that serve as building blocks for our BCC algorithms.

Euler Tour. For an unrooted tree, a directed graph T' can be obtained by replacing each edge (u, v) with two directed edges (u, v) and (v, u). We define the Euler tour of such an unrooted tree as a path of directed edges in T' that visits each edge of T' exactly once and ends at the same vertex it starts at. We use a well-known parallel algorithm technique called the *Euler Tour Technique* [22] to convert an unrooted tree into a rooted tree as well as to find the Euler tour with a specified vertex at the root. In this work, we use T to denote the rooted tree obtained by applying the Euler Tour Technique on the edges of an unrooted tree, and we represent the Euler tour by a sequence of vertices rather than edges, which is equivalent to a DFS traversal. For a vertex u in T, $\mathsf{first}[u]$ and $\mathsf{last}[u]$ denote the indices of the first and last appearance of u in the Euler tour of T. Figure 1(a) shows a rooted tree and its Euler tour. The numbers shown on the left side and right side of each vertex u denote the $\mathsf{first}[u]$ and $\mathsf{last}[u]$, respectively.

Low-High Tags. For a vertex u, let T_u denote the subtree of T rooted at u. We then write $\mathsf{low}[u]$ to denote the smallest $\mathsf{first}[\cdot]$ value of all the vertices that can be reached via a non-tree edge from vertices in T_u. Similarly, $\mathsf{high}[\cdot]$ is the maximum value. Formally, both $\mathsf{low}[\cdot]$ and $\mathsf{high}[\cdot]$ can be written as:

$$\mathsf{low}[u] = \min\{w_1[u] \mid u \in V(T_v)) \text{ where },$$
$$w_1[u] = \min\left(\{\mathsf{first}[u]\} \cup \{\mathsf{first}[u'] \mid (u, u') \in E(G) - E(T)\}\right)$$
$$\mathsf{high}[u] = \max\{w_2[u] \mid u \in V(T_v)) \textit{ where},$$
$$w_2[u] = \max\left(\{\mathsf{first}[u]\} \cup \{\mathsf{first}[u'] \mid (u, u') \in E(G) - E(T)\}\right)$$

Types of edges. The edges in a graph G can be classified into four types with respect to a rooted spanning tree T of G. An edge (u, v) that appears in T is called a *tree edge*, while any edge that does not belong to T is referred to as a *non-tree edge*. A non-tree edge (u, v) is classified as a *back edge* if an ancestral relationship exists between u and v; otherwise, we classify it as a *cross edge*. For a vertex v in T, let $\mathsf{par}(v)$ denote its parent. We classify the tree edges of T into two categories: **fence edges** and **plain edges**. A tree edge $(\mathsf{par}(v), v)$, is called a *fence edge* if for all non-tree edges $(x, y) \in E$, whenever x belongs to T_v, y belongs to $T_{\mathsf{par}(v)}$. Informally, a fence edge ensures that no edge from v's subtree escapes beyond $\mathsf{par}(v)$'s subtree. A tree edge that is not a fence edge is called a *plain edge*. For illustration purposes, fence edges and plain edges are shown in Figure 1 (b). An edge in a graph can be classified into one of the four types computationally in constant time using the following: first, last, low, and high tags. A tree edge $(\mathsf{par}(v), v)$ can be identified as a fence edge if $\mathsf{first}[\mathsf{par}(v)] \leq \mathsf{low}[v]$ and $\mathsf{high}[v] \leq \mathsf{last}[\mathsf{par}(v)]$; otherwise it is a plain edge. Similarly, a non-tree edge (u, v) is a back edge if $\mathsf{first}[u] < \mathsf{first}[v]$ and $\mathsf{last}[v] < \mathsf{last}[u]$ (or) vice versa; otherwise, it is a cross edge. For an edge (u, v) in a graph, both vertices u and v are referred to as *endpoints*, where u is referred to as the *first endpoint* and v is referred to as the *second endpoint*.

BCCs in Sparse Representation. We use two arrays, $label[\cdot]$ and $componentHead[\cdot]$ ([3], [10]), to represent the BCCs of a graph in a space-efficient manner. For a graph G and a rooted spanning tree T of G, for each non-root vertex u in T, $label[u]$ denotes a label from $\{1, \ldots, k\}$, where k denotes the number of BCCs in G. For each label $i \in \{1, \ldots, k\}$, $componentHead[i]$ indicates a special vertex called the *component head* or BCC *head*. All vertices with the same label and component head together form a BCC. An illustration of this representation is shown in Fig. 1(c) and 1(d).

3. Parallel In-Memory BCC Algorithm

Our primary objective is to develop a GPU-based parallel algorithm to solve the Biconnected Components (BCC) problem under the PEA model. As a foundational step, this section presents an in-memory parallel algorithm designed for scenarios where the input graph G fits entirely within the GPU global memory. In the subsequent section, we will propose the necessary extensions to this in-memory approach to handle Out-of-Memory (OOM) scenarios effectively.

Our algorithm draws inspiration from FAST-BCC, the current state-of-the-art multi-core parallel algorithm for BCC computation. The original FAST-BCC constructs a skeleton graph G' consisting of all plain-edges in spanning tree T and *all* cross-edges in $G - T$. However, we observe that not all cross-edges are essential for determining biconnectivity. Building upon this insight, we propose FAST-BCC-FILTER, a GPU-oriented variant of FAST-BCC that filters such redundant cross-edges while preserving correctness. In the following subsection, we discuss the algorithm in details.

A. Notation

In this paper, we focus on undirected, unweighted graphs, which we denote as $G = (V, E)$. V and E are the set of

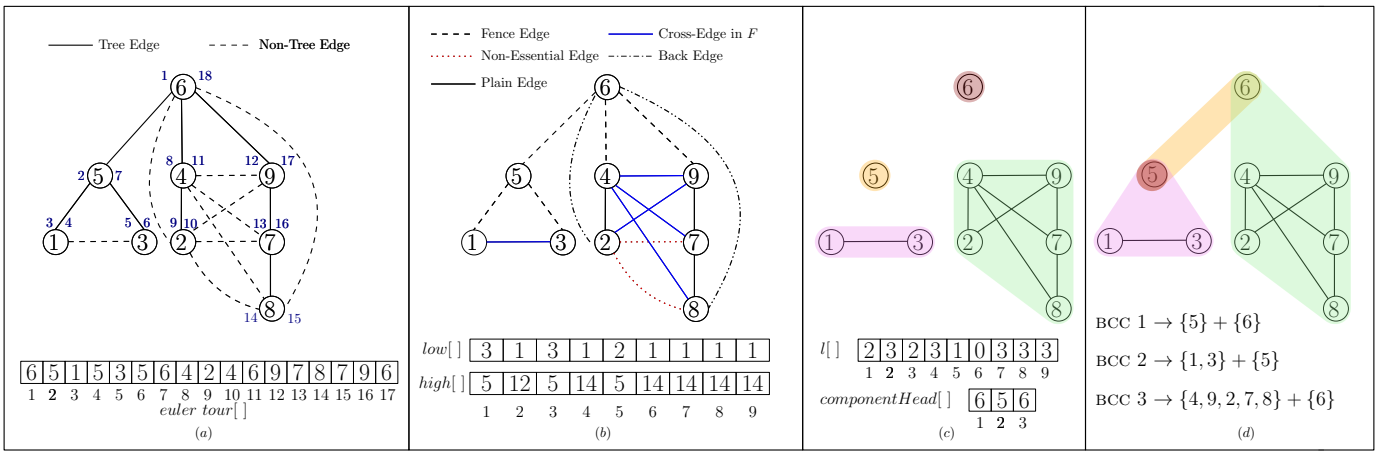

Fig. 1: **Steps of Algorithm 1 for BCC:** (a) Lines 1-2: Obtain a rooted spanning tree T of G, its Euler tour, and compute first/last tags. (b) Line-4: Compute low/high tags using all back edges and cross edges in F_c. (c) Lines 5-9: Construct a skeleton graph G' with plain edges in T and edges in F_c, find labels for connected components in G', and obtain componentHead for each label except for the label of the root vertex. (d) BCCs of G are shown in sparse representation.

Algorithm 1: FAST-BCC-FILTER ALGORITHM

Input: An undirected graph $G(V, E)$
Output: labels $l[\cdot]$ and componentHead[$\cdot$]

1 $E_T \leftarrow$ CONSTRUCT-SPANNING-TREE(G)
2 $(T, \mathsf{first}[\cdot], \mathsf{last}[\cdot]) \leftarrow$ EULER-TOUR(E_T)
3 $(\mathsf{low}[\cdot], \mathsf{high}[\cdot], F_c) \leftarrow$ COMPUTE-SF-TAGS(G)
4 $G' \leftarrow$ PLAINEDGES$(T) \cup F_c$ `//Skeleton graph`
5 $l[\cdot] \leftarrow$ CC-LABELS(G') `//Labels for vertices`
6 **for** *each vertex* $v \in V \setminus \{r\}$ **do in parallel**
7 **if** $l[v] \neq l[\mathsf{par}[v]]$ **then**
8 $componentHead[l[v]] \leftarrow \mathsf{par}[v]$

vertices and edges, and $n = |V|$ and $m = |E|$ are the number of vertices and edges, respectively. We use E_c and E_b to denote the sets of back edges and cross edges of $G - T$, respectively, where T is an arbitrary rooted spanning tree of G. Additionally, F_c denotes a spanning forest of the edges in E_c. The edges in F_c are referred to as *essential edges*.

Throughout the paper, we consider the graph to be in edge-stream (COO) format using a sequence of edges, where each edge is a pair of vertices.

B. Overview

Our proposed algorithm, FAST-BCC-FILTER involves the following five major steps to obtain BCCs of G. The first three steps, CONSTRUCT-SPANNING-TREE, EULER-TOUR for *rooting*, and COMPUTE-SF-TAGS for *Tagging* helps to construct a reduced graph (skeleton) G'. The next two steps involve applying CC-LABELS for obtaining labels to vertices in G' as per the connected components and obtaining a sparse representation of BCC.

As in FAST-BCC, we compute per-vertex tags that summarize non-tree reachability; however, unlike the standard approach that loops over *all* edges to derive low[$\cdot$] and high[$\cdot$], our method is deliberately selective. We process only the set E_b of *back edges* and a sparsified forest F_c of *cross edges*. Consequently, the skeleton G' contains all plain edges from the spanning tree together with only the cross edges retained in F_c, while necessary information from back edges is summarized implicitly through the tags. This selective approach reduces work in the final connectivity phase without sacrificing correctness, as we prove in Theorem 3.3.

C. Algorithmic Details

The BCC algorithm proceeds in five main steps. First, we construct a spanning tree from the given graph G using the connected components (CC) algorithm [28]. The CC algorithm solves the connected components problem and finds a spanning forest of a graph. Since the input graph G is connected, CC produces a spanning tree of G. The second step is meant for converting an unrooted spanning tree to a rooted spanning tree T using the EULER TOUR TECHNIQUE ([22]), and computing first[$\cdot$] and last[$\cdot$] for all vertices in T. The second step requires $O(\log n)$ depth [15, Section 3.5]. During the third step, we apply COMPUTE-SF-TAGS to construct a spanning forest F_c on all cross-edges and compute the tags low[$\cdot$] and high[$\cdot$] for all vertices using the non-tree edges in F_c and E_b. The algorithm details of COMPUTE-SF-TAGS are deferred to the next subsection. In the fourth step, we run CC-LABELS algorithm ($O(\log n)$ depth [28]) on the skeleton graph formed by plain edges in T and cross-edges in F_c. The CC-LABELS assign labels to all vertices such that all vertices in the same connected component receive the same label. Finally, in the fifth step (constant depth), we obtain a component head for each label by processing all non-root vertices of T in parallel. For a non-root vertex v, componentHead of $label[v]$ is updated with its parent. All vertices with the same label and their component head together form a BCC. Finally, we

conclude that FAST-BCC-FILTER algorithm produces BCC in sparse representation. The complete execution of the *filter* algorithm is illustrated in Figure 1.

D. COMPUTE-SF-TAGS *subroutine*

We recall that the purpose of this subroutine is to construct a spanning forest F_c on all cross-edges and compute the tags $\mathsf{low}[\cdot]$ and $\mathsf{high}[\cdot]$ for all vertices using the non-tree edges in F_c and E_b. To achieve this, we use the rooted spanning tree T represented by the parent array $\mathsf{par}[\cdot]$ and two arrays, namely $\mathsf{first}[\cdot]$ and $\mathsf{last}[\cdot]$.

The algorithm processes all non-tree edges (edges in $G-T$) in parallel and classifies them as a cross edge or back edge using the $\mathsf{first}[\cdot]$ and $\mathsf{last}[\cdot]$ arrays. It then constructs a spanning forest F_c on the identified cross edges using the CC algorithm [13]. Later, we sort all the back edges (u, v) based on their first endpoints using [19], where v is an ancestor of u. We then perform segmented min-reduction on the sorted edges by considering $\mathsf{first}[\cdot]$ values of second endpoints, where all edges with the same first endpoint are treated as segments. These two steps help to obtain $w_1[\cdot]$ by considering all edges in $F_c \cup E_b$, where $w_1[u]$ denotes the minimum of all first values of second endpoints of non-tree edges going out from the subtree rooted at T_u and $first[u]$. $w_2[\cdot]$ array is analogues to $w_1[\cdot]$, except that it focuses on maximum value. All operations require to compute the spanning forest F_c, computing $w_1[\cdot]$ and $w_2[\cdot]$ tags take $O(\log n)$ depth.

For each vertex u, all vertices in the subtree of u appear consecutively in the Euler tour $ET[\cdot]$ of T from $\mathsf{first}[u]$ to $\mathsf{last}[u]$. For each vertex u, we compute $\mathsf{low}[u]$ (minimum of all w_1 values of all vertices in subtree of u) by applying a range minimum query defined by $(\mathsf{first}[u], \mathsf{last}[u])$ on $ET_1[\cdot]$ array, where for each $1 \leq i \leq 2\times(n-1)$, $ET_1[i] = w_1[ET[i]]$. Computing $\mathsf{high}[\cdot]$ tags is similar to that of $\mathsf{low}[\cdot]$, except that we use w_2 instead of w_1, and apply range maximum query instead of range minimum query. For the Parallel Range Minimum Query, we use the sparse table approach [5], which takes $O(\log n)$ depth.

We prove the correctness of the FAST-BCC-FILTER algorithm in the next section.

E. Cut Vertices, Cut Edges and Edge Labels.

Given a graph G, FAST-BCC-FILTER algorithm (Algorithm 1) produces labels ($l[\cdot]$) for non-root vertices and component heads ($componentHead[\cdot]$) of all labels except the label of the root vertex. We now present two characterizations that help to find all cut vertices and all cut edges in the parallel setting in constant time using labels and component head arrays.

Let w be an arbitrary vertex in G and r be the root vertex of the spanning tree used in Algorithm 1. w is a cut vertex if and only if one of the following holds true. i) $w \neq r$ and w is a component head for atleast one label ii) $w \neq r$ and w is a component head for at least two labels. An edge $(par[u], u)$ is a cut edge iff the frequency of label of u in $l[\cdot]$ is exactly one. From vertex labels and component heads, we can obtain a

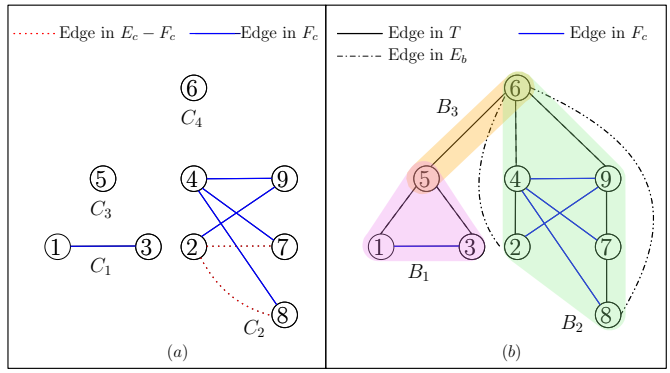

Fig. 2: Proof Intuition: T is a rooted spanning of G, E_b denote the set of back edges w.r.t T, E_c denote the set of cross edges w.r.t. T and F_c is a forest built on E_c. $C_1, \ldots, C_4$ are conneccted components in F_c. B_1, B_2, B_3 are biconnected components in $T \cup E_b \cup F_c$. Edges in $E_c - F_c$ are non-essential for BCC.

BCC label for an edge (u, v) in G in constant time as follows: If $l[u] = l[v]$, then edge label is $l[u]$; If v is component head of $l[u]$, then edge label is $l[u]$; Otherwise, edge label is $l[v]$.

F. Correctness of Fast-BCC Filter Algorithm

We first describe the intuition of Lemma 3.2 and Theorem 3.3 using an example in Figure 2, followed by a formal proof of the correctness of Algorithm 1. We can observe that for every connected component C in F_c shown in Figure 2(a), there exists a biconnected component B in $T \cup F_c \cup E_b$ shown in Figure 2(b) such that all edges of C appear in B. Let x and y be two vertices in a connected component C. Then there exists a BCC B such that x and y belong to B. Let (x, y) be an edge in $E_c - F_c$. Since vertices x and y appeared in B, adding edge (x, y) to $T \cup F_c \cup E_b$ does not merge the existing biconnected components. Therefore, removal of such edges from G does not disturb the biconnectivity. These observations are formally proved in the following two lemmas. We now proceed to prove the correctness of Algorithm 1.

Fact 3.1. *Each edge in a graph belongs to exactly one bcc.*

Lemma 3.2. *Each connected component of F_c is in one biconnected component of $T \cup F_c \cup E_b$.*

Proof: Let C be a connected component of F_c. Note that C is a tree, and each edge in C is a cross edge in $G - T$. We apply induction on the number of edges in C and show that all edges in C belong to the same BCC in $T \cup C \cup E_b$. Base condition is when there is only a single edge in C. By Fact 3.1, that edge is in a biconnected component of $T \cup C \cup E_b$.

An edge $e(u, w)$ exists in C, such that e is adjacent to a leaf node, and all edges in $C - e$ remain connected since C is a tree. Due to the inductive hypothesis, we know that all edges in $C - e$ belong to the same bcc in $T \cup C \cup E_b$. We can find an edge in $C - e$ which is adjacent to e, say $e' = (w, v)$. We know that any edge in F_c is a cross edge, and there is no ancestral relationship between the endpoints of that edge in

T. Hence, there are no ancestral relationships between u and w, and v and w in tree T.

If there is no ancestral relationship between u and v, then the paths in T from u to the least-common ancestor of u and v, denoted $lca(u, v)$ and from v to $lca(u, v)$ plus the edges (u, w) and (v, w) in C form a simple cycle. If there is some ancestral relationship between u and v, then the paths in T between v and u, the edge (v, w) and the edge (u, w) form a simple cycle. In both cases, e' and e are in a cycle. It follows that e' and e belong to a BCC in $T \cup C \cup E_b$. Consequently, all edges in C appear in the same BCC in $T \cup C \cup E_b$.

Due to the observation that adding edges to a graph only merges BCCs and by Fact 3.1, we conclude that all edges of C are in one BCC of $T \cup F_c \cup E_b$. ■

Theorem 3.3. *Two vertices are biconnected in G if and only if they are biconnected in $T \cup F_c \cup E_b$.*

Proof: Consider a connected component C formed with edges in E_c. Let T' be a tree in F_c such that T' is a spanning tree of C. Let us consider an edge (x, y) in $C - T'$. By Lemma 3.2, there exists a biconnected component B in $T \cup F_c \cup E_b$ such that $V(T') \subseteq V(B)$. Consequently, x and y belong to $V(B)$ and thus are part of a cycle in $T \cup F_c \cup E_b$. Adding the edge $e(x, y)$ to the corresponding biconnected component containing x and y does not alter the graph's biconnectivity. Therefore, the biconnectivity of G (that is $T \cup E_c \cup E_b$), is equivalent to that of $T \cup F_c \cup E_b$. ■

By Theorem 3.3, we know that applying a biconnected components algorithm on $T \cup F_c \cup E_b$ is sufficient to find the biconnected components of G. The steps described in FAST-BCC-FILTER algorithm (Algorithm 1) are responsible for extracting $T \cup F_c \cup E_b$ from G, and running FAST-BCC algorithm [10] on $T \cup F_c \cup E_b$.

The task of applying FAST-BCC algorithm on $T \cup F_c \cup E_b$ boils down to the task of obtaining a skeleton graph of G with PLAINEDGES$(T) \cup F_c$, which is shown in Line 4. Most importantly, the edges in $E_c - F_c$ do not contribute to the biconnectivity due to Theorem 3.3, and hence we ignore them while obtaining the skeleton graph in our FAST-BCC-FILTER algorithm. All the rest of the steps in both FAST-BCC and FAST-BCC-FILTER algorithms are essentially the same. Thus, FAST-BCC-FILTER algorithm correctly identifies the BCCs of a given graph.

Sparse Skeleton. $T \cup E_c$ is a skeleton graph of G in FAST-BCC algorithm, whereas PLAINEDGES$(T) \cup F_c$ is a sparse skeleton graph of G in our FAST-BCC-FILTER algorithm. The number of edges in PLAINEDGES$(T) \cup F_c$ is at most $2n - 2$, and thus the skeleton graph in our approach consumes $O(n)$ space. Most importantly, the skeleton graph is sufficient to obtain the BCCs in sparse representation.

4. BCC Algorithm in PEA Model

The parallel in-memory algorithm to compute BCC, shown in Algorithm 1, requires the whole graph to fit in GPU memory. In this section, we make the necessary changes to this algorithm to obtain the PEA-BCC algorithm, that can handle massive graphs exceeding the global memory of a GPU.

Formally, let $G = (V, E)$ be a graph with $n = |V|$ vertices and $m = |E|$ edges. We recall that the PEA model assumes a system with two memory tiers: a CPU memory, $M_{\text{CPU}} = \Theta(m)$, sufficient to store the entire edge list, and a GPU memory, M_{GPU}, of size $O(n)$ such that a subset of edges and all required per-vertex arrays can reside on the device simultaneously. We now detail the proposed changes.

PEA-BCC Algorithm. The primary challenge arises in two specific procedures: CONSTRUCT-SPANNING-TREE(G) and COMPUTE-SF-TAGS(G), corresponding to Lines 1 and 3 of Algorithm 1 respectively. As these steps require iterating over the entire edge list, we detail their implementation with respect to the PEA model below. The remaining steps in Algorithm 1 operate within $O(n)$ space and do not require the entire input graph to be stored in GPU memory; thus, they can be executed in the PEA model without modification.

Initially, all edges $(e_1, \ldots, e_m)$ of G appear in CPU. We partition the edges of G into equal-sized batches $B_1, B_2, \ldots, B_k$ in an arbitrary way, in CPU. The last batch may have a smaller size depending on the number of edges and the batch size. The batch size is a hyperparameter chosen in such a way that one batch of edges and the auxiliary space used by the data structures in the GPU together is within the available memory of the GPU.

Pass-1: CONSTRUCT-SPANNING-TREE(G) in the PEA model. Initially, an empty spanning forest of G lies in the device. For each batch B_i of edges in G, we copy B_i from host to device, apply the incremental spanning-forest algorithm [13] on B_i to update the partial spanning forest in GPU. After all batches are processed, a spanning forest F of G is ready in the device. We then apply the Euler Tour Technique (ETT) on F to obtain a rooted spanning forest, which can be represented by $\mathsf{parent}[\cdot]$ array. Addtionally, ETT computes the $\mathsf{first}[\cdot]$ array and $\mathsf{last}[\cdot]$ array.

Pass-2: COMPUTE-SF-TAGS(G) in PEA model. For each batch B_i of edges, we first copy B_i from host to device, and then classify its edges as tree, back, or cross edges. The identified cross edges are used to incrementally build a spanning forest F_c, using the external algorithm discussed earlier. For *back edges* in B_i, we update the w_1 and w_2 arrays exactly as defined in the previous section. These arrays, as discussed earlier, track the minimum and maximum discovery times reachable via non-tree edges from each vertex's subtree in T, respectively. After all batches have been processed, we loop over the edges in the forest F_c to finalize the w_1 and w_2 arrays. Then *low*$[\cdot]$ and *high*$[\cdot]$ are computed from these aggregated arrays using the range minimum/maximum query (RMQ) method as described earlier.

Depth and Space Analysis. From the above description, it is clear that our PEA-BCC algorithm is a two-pass algorithm, as each edge is copied twice from CPU to GPU. Processing a batch of edges in GPU, requires $O(\log n)$, as $O(\log m) = O(\log n)$ in both passes. Thus, the total depth is bounded by $O(k \log n)$, where k denotes the number of batches. All the data structures

used in our algorithm in GPU consume $O(n)$ space, as each batch fits in GPU and GPU holds $O(n)$ space.

We note that while the algorithm and our implementation naturally handle disconnected graphs, for the sake of simplicity, the pseudo-code assumes that the input graph is connected.

I/O Complexity Analysis. Our algorithm transfers the entire edge list from CPU memory to GPU memory exactly twice: once for constructing an arbitrary rooted spanning tree and once for tag computation. Therefore, the theoretical I/O complexity of the algorithm is $O(m/B)$, where m is the number of edges and B is the batch size.

5. Implementation Details

This section outlines the key implementation decisions and optimizations used in both our in-memory and external memory BCC implementations.

Connectivity and Spanning Forest. Connectivity plays a crucial role in both the construction of the auxiliary spanning tree and in identifying connected components during the final phase. We adopt a GPU-optimized connected components (CC) implementation by Hong et al. [13], referred to as GConn, which is the current state-of-the-art on GPUs. From various techniques proposed in their work, we choose k-out sampling for our GPU setting due to its simplicity and general applicability across graph types. We acknowledge that alternative sampling techniques could yield further improvements, depending on the graph type. For the CC-LABELS (Line 5) procedure, we do not apply any sampling, since the input graph at this stage is already sparse (tree + small set of cross edges), and a full traversal is feasible with limited overhead.

In the PEA-BCC implementation, for constructing the external-memory spanning forest, we use the incremental spanning forest algorithm presented in the same Gconn paper. We use copy-computation discussed later in this section.

Eulerian Tour. We use the ET algorithm proposed by Polák et al. [22], with minor modifications to support forests.

Tag computation. For computing w_1 and w_2 tags, we use atomic operations in our implementations, due to the performance benefit when compared to applying segmented min reduction on sorted edges.

Range Minimum Query (RMQ). A standard parallel range minimum query implementation using the sparse table approach [4] requires $O(\log n)$ preprocessing time and $O(n \log n)$ space complexity, enabling range-minimum queries in $O(\log n)$ time. To reduce the space overhead from $O(n \log n)$ to $O(n)$, while maintaining the same time complexity, we adopt the two-level sparse table technique proposed in [5]. This approach achieves $O(\log n)$ preprocessing and query time while using only linear space.

I/O and Computation Overlap. To ensure maximum hardware utilization, we use **Double Buffering with Pinned Memory.** We employ double buffering on pinned host memory to overlap data transfer and computation. This technique enables simultaneous GPU computation on one buffer while the next batch of data is being transferred, resulting in a 15-20% performance improvement over naive implementations. To achieve the above copy-computation pipeline, we utilize 2 CUDA streams to overlap data transfers with kernel execution. This technique enables the CPU to read the next batch of data from the CPU while the GPU is actively processing the current batch, leading to a performance improvement.

6. Experimental Results

In the following subsections, we compare the performance of our in-memory FAST-BCC-FILTER and PEA-BCC algorithms' runtime against state-of-the-art implementations from the literature. All numbers reported in this section are the median of five runs. To highlight complementary strengths, we benchmark on two NVIDIA GPU platforms, NVIDIA L40 (48 GB memory) and NVIDIA L4 (24 GB memory). The larger memory of L40 highlights the strengths of our in-memory algorithm, while the smaller memory of L4 emphasizes the advantages of our external-memory (PEA) design on memory-constrained hardware. We begin with overall speedup results, then present a kernel-level breakdown to identify bottlenecks, and finally study the scalability of the external algorithm.

Baselines. We evaluate our proposed solution against three primary baselines: WK-BCC (2017) [31], WK-BCC (2018) [16], and the more recent FAST-BCC (2023) [10]. The former two represent the state-of-the-art in GPU-based algorithms, while FAST-BCC is the current state-of-the-art for multi-core parallel BCC computation. We exclude the SM-14 algorithm from this comparison, as both WK-BCC and FAST-BCC have consistently outperformed it in prior evaluations. We also omit comparisons with general-purpose graph processing frameworks (such as *Gunrock*, *cuGraph*, or GBBS), as they either lack native support for BCC primitives or are previously outperformed by the specialized baselines.

To ensure a fair comparison, we implemented the FAST-BCC algorithm on the GPU using the GPU-specific optimizations detailed earlier in the implementation section. Regarding WK-BCC, since the source code was not publicly accessible, we re-implemented the algorithm following the specifications detailed in [31], [16]. To facilitate reproducibility and future comparisons, we have made our implementation publicly available.[†] For the multi-core reference baseline, we utilized the official FAST-BCC source code from its public repository.

In the next section, we describe the experimental setup and the characteristics of the datasets used for evaluation.

A. Experimental setup

Our GPU evaluation is performed on two machines. The first uses an NVIDIA L40S, an Ada-generation GPU with 48 GB GDDR6 VRAM providing 864 GB/s peak memory bandwidth and 142 Streaming Multiprocessors (SMs). It is connected to a dual-socket Intel Xeon Platinum 8480+ host (112 cores / 224 threads, 3.8 GHz boost) with 210 MB L3 cache and 256 GB DDR5 RAM.

The second system contains an NVIDIA L4 GPU, also based on the Ada architecture, with 24 GB GDDR6 VRAM, 300 GB/s

[†]https://github.com/Abhijeetkumar96/FAST-BCC-on-GPUs

TABLE II: Dataset statistics showing the number of vertices (n), edges (m), and approximate diameter (D). *WK*-17 refers to the BCC algorithm by Wadekar and Kothapalli [31]. *WK-18* refers to the sampling-based BCC algorithm [16]. *spd* denotes the speed-up achieved by our best implementation over the best WK-BCC, when the graph fits in GPU. Otherwise, speed-up is obtained with respect to the best multicore CPU baseline, FAST-BCC. *OOM* indicates out-of-memory cases. Values highlighted in blue with asterisks represent instances where our FAST-BCC-FILTER failed, and the corresponding PEA results are reported instead. When data fits our best implementation refers to FAST-BCC-FILTER, otherwise it refers to PEA-BCC. All reported timings are in **milliseconds (ms)**.

Datasets		n	m	D	L40 (48 GBs)						L4 (24GBs)						Note
					WK-18	WK-17	Fast-BCC CPU	Fast-BCC GPU	Ours (best)	spd.	WK-18	WK-17	Fast-BCC CPU	Fast-BCC GPU	Ours (best)	spd.	
Web Graphs	IN	7.41M	151M	306	182	175	197	60	29	6	302	273	523	123	69	4	indochina-2004
	AR	22.7M	1.11B	539	OOM	556	536	199	94	6	OOM	1310	1263	426	229	6	arabic-2005
	UK-05	39.5M	1.57B	12k	OOM	4117	834	254	160	26	OOM	OOM	1674	604	406	4	uk-2005
	WB	118.1M	1.72B	755	OOM	1797	2440	477	387	5	OOM	OOM	3592	OOM	1488*	2	webbase-2001
	IT	41.3M	2.05B	986	OOM	1658	965	365	170	10	OOM	OOM	1941	786	420	4	it-2004
	GW	50.6M	3.62B	133	OOM	OOM	1232	675	222	6	OOM	OOM	2200	OOM	1600*	2	GAP-web
	SK	50.6M	3.62B	132	OOM	OOM	1213	684	222	5	OOM	OOM	2580	OOM	1124*	2	sk-2005
	UKU	133.6M	9.33B	2M	OOM	OOM	3073	OOM	3474*	0.88	OOM	OOM	6636	OOM	4474*	1.5	uk_union_2006_07
Social Networks	WT	1.79M	25.4M	8	87	69	46	8	5	13	170	101	144	21	14	7	wiki_topcats
	SO	2.60M	28.2M	23K	959	471	81	12	9	54	1473	768	249	34	22	35	sx_stackoverflow
	LJ	5.36M	49.5M	82	212	42	113	21	18	2	410	87	321	66	53	1.7	ljournal_2008
	HW	1.14M	56.4M	44K	1476	852	51	14	4.5	191.1	1921.1	1014	106	31	11	90	hollywood_2009
	CO	3.07M	117M	8	106	80	129	32	12	6.4	240	172	364	85.5	34	5.1	com_Orkut
	GT	61.6M	2.44B	17	OOM	7625	3307	2381	711	11	OOM	OOM	9340	5176	1866.5	5.1	GAP_twitter
	CF	65.6M	3.61B	24	OOM	OOM	3760	3905	1578	2.4	OOM	OOM	16042	OOM	7179*	2.2	com_Friendster
Road Net-works	GO	11.5M	12.4M	4K	330.4	208	230	33.1	33.4	6.22	704.28	466	638	85.78	85.92	5.42	germany_osm
	RU	23.9M	28.9M	6K	1005	505	408	65.1	66	7.7	3270	2418	893	178	183	13.2	road_usa
	EO	50.9M	54.1M	16K	5445	5701	834	148	146	38	OOM	15615	2224	378	380	41.1	europe_osm
Citation & Biological Networks	AS	1.70M	11.1M	761	62	35	38	5.1	4.6	7.6	109	56	117	14.1	10.8	5.2	as_Skitter
	DBLP	540K	15.2M	15	23	6	18	3.4	3.05	2	57	10	54	7.8	4.2	2.3	coPapersDBLP
	CI	430K	16.0M	25	26	6	21	4	3.1	2	73.6	10	58	8.6	4.2	2.4	coPapersCiteseer
	ML	30.2M	6.68B	25K	OOM	OOM	2874	OOM	228	12.6	OOM	OOM	15K	OOM	5280*	2.9	MOLIERE_2016
	AG	184M	11.6B	14	OOM	OOM	14949	OOM	3434*	4.35	OOM	OOM	48234	OOM	6879.5*	7	AGATHA_2015
Synthetic & Random	K18	260K	10.6M	52K	1212	774	20	3.8	2.2	348.5	1666	721	39	11.5	8.32	87	kron_g500_logn18
	K19	520K	21.9M	115K	3364	1898	28	13.1	3.01	630	4561	2536	66	21.15	13.43	189	kron_g500_logn19
	K20	1.05M	44.9M	253K	9077	5288	42	23	14	382	12216	5658	110	33	15.1	375	kron_g500_logn20
	RGG23	8.39M	63.5M	1K	216	185	261	38.1	35	5.3	516	387	705	82	75	5.2	rgg_n_2_23_s0
	K21	2.10M	91.6M	526K	20647	11776	95	38	23	517	31137	16809	198	65	29	584	kron_g500_logn21
	RGG24	16.8M	133M	2K	467	394	545	78	70	5.6	1586	1326	1633	159	146	9	rgg_n_2_24_s0
	GK	134.2M	4.37B	9	OOM	OOM	6191	OOM	824	7.5	OOM	OOM	19955	OOM	8105*	2.4	GAP_kron

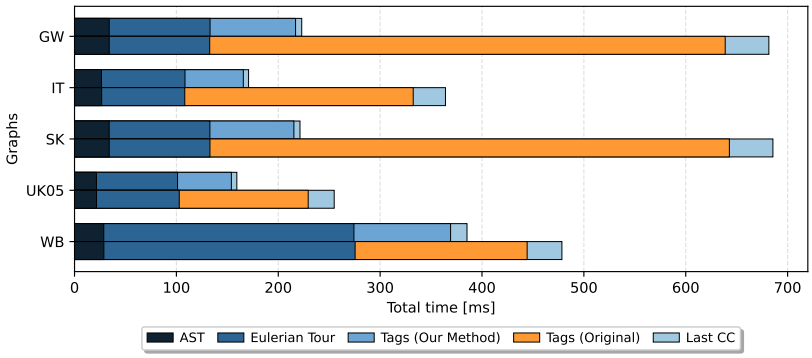

Fig. 3: Performance benefit of using the Filter

peak memory bandwidth, and 24 SMs, connected via PCIe Gen 4 ×16 to a dual-socket AMD EPYC 7742 (64 cores per socket, 2.25 GHz) host with 512 GB DDR4 RAM. All implementations were compiled using `NVCC v12.9` with the `-O3` flag, targeting compute capability 8.9 (`--sm_89`).

Graph Data. To evaluate the performance and scalability of our BCC algorithms, we conduct experiments on a diverse set of over 30 real-world graphs with varying sizes and structural properties. Our objective is to validate the robustness of our implementation across a wide spectrum of graph topologies.

The dataset includes both low-diameter graphs, such as web and social networks, and high-diameter graphs, such as road networks. The scale of these graphs ranges from those with over 30 million edges to extremely large networks, including 9 graphs with more than 1 billion edges. As a standard preprocessing step, all graphs were symmetrized to ensure they are undirected and connected as both the gpu baselines (WK-17] & WK-18) require a connected graph.

A detailed summary of the input graphs, along with absolute running times across all datasets, is reported in Table II.

B. Performance Analysis of In-memory BCC Algorithm

Our **FAST-BCC-FILTER** algorithm consistently outperforms all baselines across all graphs when the data fits entirely in GPU memory. On the **L40s** machine, our FAST-BCC-FILTER achieves an average speedup of 71× over the faster of the two WK-BCC variants [31], [16]. Relative to the CPU-based FAST-BCC, our GPU implementation of FAST-BCC attains an

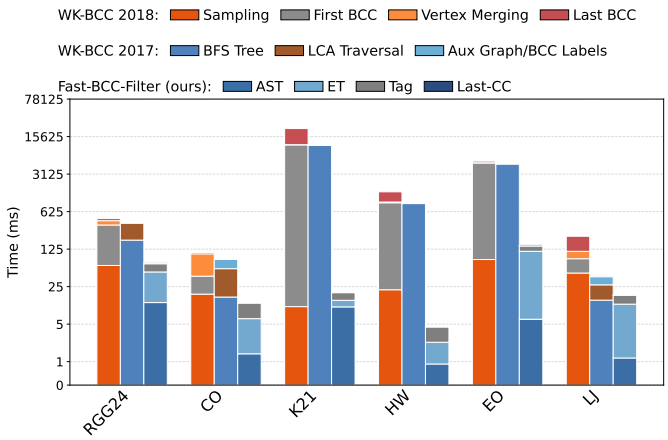

Fig. 4: Runtime breakdown of the WK-BCC algorithms compared to our FAST-BCC-FILTER

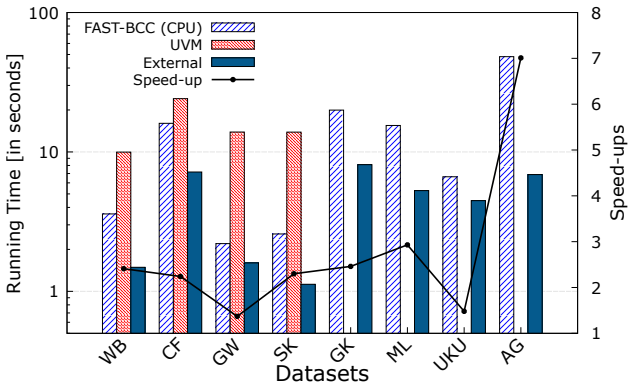

Fig. 5: Running times of various BCC algorithms.

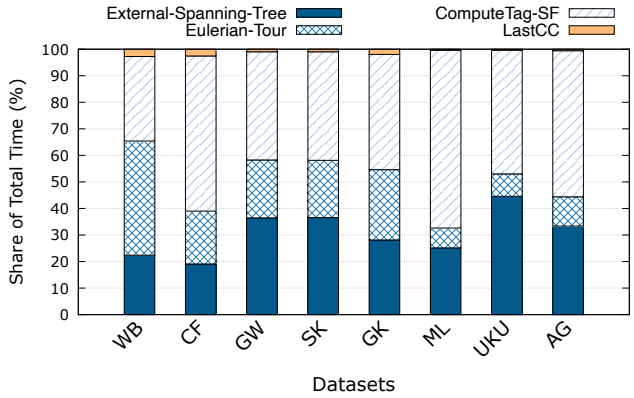

Fig. 6: Profiling of various steps of PEA-BCC.

average speedup of $4.7\times$, while our FAST-BCC-FILTER version improves this speedup to $7.2\times$, as summarized in Table II.

To better understand the performance gain of our GPU implementation over the WK-BCC variants, we analyze the performance breakdown presented in Fig. 4. Since WK-BCC fails on the very large datasets due to out-of-memory (OOM) errors, we restrict this comparison to the largest graphs where both implementations could successfully execute.

Across all such instances, our FAST-BCC-FILTER consistently outperforms the baselines. The bottleneck for WK-17 lies in the BFS tree construction. In fact, for most graphs, BFS construction alone consumes 70–80% of WK-17's total runtime, while our entire algorithm completed within that time. Regarding WK-18, our profiling reveals that while sampling itself is fast ($\sim$9% of runtime), the subsequent processing remains heavy. Specifically, both the BCC computation consumed roughly 50% and 26% of the total time, respectively, largely because WK-18 reliance on WK-17.

Similarly, on the **L4** machine, despite its lower memory capacity and bandwidth, we observe significant performance gains in both of our GPU variants. The FAST-BCC and FAST-BCC-FILTER achieve average speedups of *6x* and *9x* over the multicore CPU implementation, respectively, and an average speedup of *55x* over the faster variant of the WK-BCC baseline.

Notably, the L40's larger memory allowed it to process all datasets, barring only the largest ones like *UKC and AG*. Despite its smaller memory, the L4 performed on par with the L40 on the majority of compatible datasets.

1) Performance Benefit of the Filter Algorithm: Building on the above discussion, we now analyze the advantages introduced by our new **Filter** variant over the original GPU implementation. As shown in Figure 3, the running time of the first two steps in both algorithms remains identical. The key performance gain arises from a significant reduction in the *tag computation* phase. Instead of computing w_1 and w_2 values for all edges in the graph, the filter algorithm computes them for only a small subset of edges, which substantially lowers the computational workload. Furthermore, the CC-LABELS procedure operates on a compact skeleton graph of size $O(n)$, which is significantly smaller and faster to process than the original graph of size $O(m)$.

However, an exception to this performance gain is on **road networks**. Compared to our original GPU implementation, the **FAST-BCC-FILTER** is faster on almost all graph types except for these road networks. Road networks are inherently sparse, and experimentally, we found the number of cross-edges to be negligible across multiple runs. Consequently, the filter has very few edges to prune, and the minor overhead introduced by the filtering and spanning forest construction phase slightly outweighs its benefits on these specific graph structures.

C. Performance Analysis of PEA-BCC Algorithm

In the Out-of-memory setting, two common fallback solutions are typically considered: (i) using Unified Virtual Memory (UVM), or (ii) switching to a CPU-based algorithm. While UVM simplifies development by enabling a shared address space between the CPU and GPU, its performance degrades significantly when the working set exceeds available GPU memory, due to excessive page migrations. On the other hand, CPU-based algorithms avoid GPU memory limitations altogether but can place a substantial load on an already busy CPU, and often fail to match GPU throughput. While we acknowledge that our external-memory GPU algorithm and

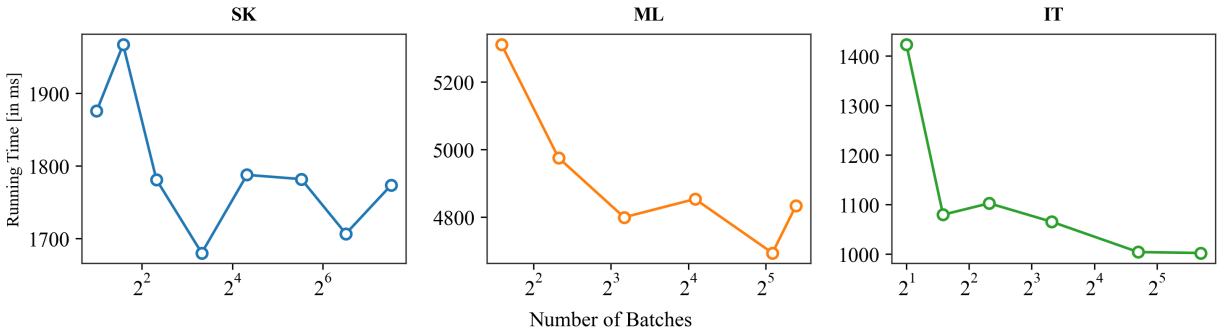

Fig. 7: Scalability of PEA-BCC on varying the number of batches

CPU-based implementations are not directly comparable, we present this comparison for reference and completeness.

Most of our datasets were processed successfully on the larger NVIDIA L40 (48 GB) GPU. However, several large graphs resulted in out-of-memory (OOM) errors even with our FILTER-BCC algorithm on the smaller L4 GPU (24 GB). In these cases, our proposed (PEA-BCC) proved crucial. As highlighted by the blue asterisks (*) in Table II, PEA-BCC successfully processed even the largest datasets, where both in-core GPU and UVM-based approaches failed. On average, we observe a 2.7× **speedup** over the fallback CPU-based implementation across those datasets as highlighted in Fig 5.

Fig. 5 further illustrates the limitations of UVM on large graphs. While UVM provides convenience through memory abstraction, it performs poorly when datasets exceed GPU memory. In such cases, UVM triggers frequent page migrations between host and device memory. These migrations introduce significant overhead, often leading to memory allocation failures or severe performance slowdowns. In our experiments, datasets with more than a billion edges could not be processed reliably using UVM on a single GPU, either due to allocation errors or unacceptable runtimes.

The fundamental issue lies in the fact that UVM operates at page granularity, while graph algorithms typically exhibit irregular and fine-grained access patterns. This mismatch undermines GPU page migration heuristics. When the working set substantially exceeds GPU memory, the system spends more time handling page faults and data transfers than performing useful computation.

1) Performance analysis of PEA-BCC*:* Fig 6 illustrates the kernel-level breakdown of execution time across different graph datasets. The performance profile reveals that the tag computation kernel consumes 40-50% of total runtime due to batch processing. Unlike the in-memory variant, PEA-BCC introduces additional overhead from batch processing and inter-batch synchronization. However, this overhead is offset by the ability to process graphs that would otherwise exceed GPU memory capacity.

2) Scalability of PEA-BCC*:* To study the scalability of PEA-BCC algorithm under varying memory constraints, we conducted experiments by adjusting the batch size to simulate GPUs with different memory capacities. Remarkably, we successfully processed several large graphs on configurations with as little as 5 GB of GPU memory, as shown in Fig 7, even though these graphs originally required 20 GB or more under the in-memory implementation.

Fig 7 further illustrates the relationship between number of batches and execution time. Contrary to intuition, the maximum batch size (in terms of edges processed per batch) does not necessarily yield optimal performance. We identified that smaller batches enable better overlap between CPU-GPU data transfers and GPU computation. For instance, on graph *MOLIERE-2016*, using 42 batches (approx. 80M edges per batch) achieved 9% better performance than using the maximum possible batch size (1.2B edges), as highlighted in Fig 7.

While graphs with smaller vertex sets showed clear scalability gains, the benefits for large-vertex graphs were limited. In such cases, varying the batch size provides limited scalability benefits, as the fixed memory requirement for vertex data constrains the minimum GPU memory needed. Nevertheless, the results demonstrate that PEA-BCC can maintain stable performance across a wide spectrum of memory configurations.

7. Conclusion

In this paper, we have provided a characterization to identify certain edges with respect to an arbitrary spanning tree of a graph whose removal does not affect the biconnected components. This characterization helped design a GPU-based parallel algorithm to find biconnected components, even in the OUT-OF-MEMORY setting. Our algorithms outperform the state-of-the-art multi-core and GPU algorithms. In future, we plan to explore this line of work by considering heterogeneous execution models targeting the out-of-memory setting.

8. Acknowledgement

This work was partially supported by a grant from the Department of Science and Technology (DST), India, under the Core Research Grant Scheme (Ref. No. CRG/2023/005225). The authors thank the anonymous reviewers for their valuable feedback and Chinmaya Bhat K K for assisting with the figures presented in this manuscript.

References

[1] L. Arge, M. T. Goodrich, M. J. Nelson, and N. Sitchinava, "Fundamental parallel algorithms for private-cache chip multiprocessors," in *SPAA 2008: Proceedings of the 20th Annual ACM Symposium on Parallelism in Algorithms and Architectures, Munich, Germany, June 14-16, 2008*, F. M. auf der Heide and N. Shavit, Eds. ACM, 2008, pp. 197–206.

[2] L. Arge, M. T. Goodrich, and N. Sitchinava, "Parallel external memory graph algorithms," in *24th IEEE International Symposium on Parallel and Distributed Processing, IPDPS 2010, Atlanta, Georgia, USA, 19-23 April 2010 - Conference Proceedings*. IEEE, 2010, pp. 1–11.

[3] N. Ben-David, G. E. Blelloch, J. T. Fineman, P. B. Gibbons, Y. Gu, C. McGuffey, and J. Shun, "Implicit decomposition for write-efficient connectivity algorithms," in *IPDPS*, 2018, pp. 711–722.

[4] O. Berkman and U. Vishkin, "Recursive star-tree parallel data structure," *SIAM Journal on Computing*, vol. 22, no. 2, pp. 221–242, 1993.

[5] G. E. Blelloch, J. T. Fineman, Y. Gu, and Y. Sun, "Optimal parallel algorithms in the binary-forking model," in *SPAA*, 2020, pp. 89–102.

[6] U. Brandes and D. Wagner, "Analysis and visualization of social networks," in *Graph drawing software*. Springer, 2004, pp. 321–340.

[7] M. Chaitanya and K. Kothapalli, "A simple parallel algorithm for biconnected components in sparse graphs," in *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. IEEE, 2015, pp. 395–404.

[8] J. Cheriyan and R. Thurimella, "Algorithms for parallel k-vertex connectivity and sparse certificates," in *Proceedings of the twenty-third annual ACM Symposium on Theory of Computing*, 1991, pp. 391–401.

[9] G. Cong and D. A. Bader, "An experimental study of parallel biconnected components algorithms on symmetric multiprocessors (SMPs)," in *IPDPS*, 2005.

[10] X. Dong, L. Wang, Y. Gu, and Y. Sun, "Provably fast and space-efficient parallel biconnectivity," in *PPoPP*, M. M. Dehnavi, M. Kulkarni, and S. Krishnamoorthy, Eds., 2023, pp. 52–65.

[11] Z. Galil, G. F. Italiano, and N. Sarnak, "Fully dynamic planarity testing with applications," *Journal of the ACM (JACM)*, vol. 46, no. 1, pp. 28–91, 1999.

[12] P. Gera, H. Kim, P. Sao, H. Kim, and D. Bader, "Traversing large graphs on gpus with unified memory," *Proceedings of the VLDB Endowment*, vol. 13, no. 7, pp. 1119–1133, 2020.

[13] C. Hong, L. Dhulipala, and J. Shun, "Exploring the design space of static and incremental graph connectivity algorithms on gpus," in *PACT*, V. Sarkar and H. Kim, Eds. ACM, 2020, pp. 55–69.

[14] J. Hopcroft and R. Tarjan, "Efficient algorithms for graph manipulation," *Communications of the ACM*, vol. 16, no. 6, pp. 372–378, 1973.

[15] J. JáJá, *Parallel Algorithms*. Addison-Wesley, 1992.

[16] K. Kothapalli and M. Wadwekar, "Expediting parallel graph connectivity algorithms," in *2018 IEEE 25th International Conference on High Performance Computing (HiPC)*. IEEE, 2018, pp. 72–81.

[17] A. Kyrola, G. E. Blelloch, and C. Guestrin, "Graphchi: Large-scale graph computation on just a PC," in *USENIX*, 2012, pp. 31–46.

[18] S. Min, V. S. Mailthody, Z. Qureshi, J. Xiong, E. Ebrahimi, and W. Hwu, "EMOGI: efficient memory-access for out-of-memory graph-traversal in gpus," *Proc. VLDB Endow.*, vol. 14, no. 2, pp. 114–127, 2020.

[19] NVIDIA Corporation, "CUB: DeviceRadixSort API," 2024, accessed: 2025-02-28. [Online]. Available: https://nvidia.github.io/cccl/cub/api/structcub_1_1DeviceRadixSort.html

[20] R. Panja and S. S. Vadhiyar, "Hypar: A divide-and-conquer model for hybrid CPU–GPU graph processing," *Journal of Parallel and Distributed Computing*, vol. 132, pp. 8–20, 2019.

[21] E. Papadopoulou, "Net-aware critical area extraction for opens in vlsi circuits via higher-order voronoi diagrams," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 5, pp. 704–717, 2011.

[22] A. Polak, A. Siwiec, and M. Stobierski, "Euler meets GPU: practical graph algorithms with theoretical guarantees," in *IPDPS*, 2021, pp. 233–244.

[23] J. H. Reif, "Depth-first search is inherently sequential," *Information Processing Letters*, vol. 20, no. 5, pp. 229–234, 1985.

[24] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu, "The ubiquity of large graphs and surprising challenges of graph processing: extended survey," *The VLDB journal*, vol. 29, pp. 595–618, 2020.

[25] S. Sakr, A. Bonifati, H. Voigt, A. Iosup, K. Ammar, R. Angles, W. Aref, M. Arenas, M. Besta, P. A. Boncz *et al.*, "The future is big graphs: a community view on graph processing systems," *Communications of the ACM*, vol. 64, no. 9, pp. 62–71, 2021.

[26] J. M. Schmidt, "A simple test on 2-vertex-and 2-edge-connectivity," *Information Processing Letters*, vol. 113, no. 7, pp. 241–244, 2013.

[27] D. Sengupta, S. L. Song, K. Agarwal, and K. Schwan, "Graphreduce: processing large-scale graphs on accelerator-based systems," in *Proc. ACM SC*, 2015, pp. 1–12.

[28] Y. Shiloach and U. Vishkin, "An O(log n) parallel connectivity algorithm," *J. algorithms*, vol. 3, pp. 57–67, 1982.

[29] G. M. Slota and K. Madduri, "Simple parallel biconnectivity algorithms for multicore platforms," in *2014 21st International Conference on High Performance Computing (HiPC)*. IEEE, 2014, pp. 1–10.

[30] R. E. Tarjan and U. Vishkin, "An efficient parallel biconnectivity algorithm," *SIAM J. Comput.*, vol. 14, no. 4, pp. 862–874, 1985.

[31] M. Wadwekar and K. Kothapalli, "A fast GPU algorithm for biconnected components," in *2017 Tenth International Conference on Contemporary Computing (IC3)*. IEEE, 2017, pp. 1–6.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

9. Overview of Contributions and Artifacts

A. Paper's Main Contributions

C_1 We propose the first GPU-based algorithm for computing biconnected components (BCCs) that operates on an arbitrary spanning tree, completely eliminating the reliance on BFS traversal and its associated sequential bottlenecks.

C_2 We present a GPU-based parallel algorithm, FAST-BCC-FILTER, to compute the Biconnected components in an undirected graph.

C_3 We extended the FAST-BCC-FILTER algorithm to the out of memory setting, which is capable of processing massive graphs that exceed available device memory.

C_4 We demonstrate that our implementation achieves an average speedup of 71× over prior state-of-the-art GPU baselines (WK-17 and WK-18) in the in-memory setting. For large-scale datasets where existing GPU approaches fail due to memory limitations, our out-of-memory (OOM) implementation outperforms the strongest CPU baseline (FAST-BCC) by an average of 2.7×.

B. Computational Artifacts

A_1 https://doi.org/10.5281/zenodo.18669968

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3 and C_4	Table II and Figures 3-7

10. Artifact Identification

B_1 Computational Artifact $A1$

Relation To Contributions: This artifact contains the C++/CUDA source code for the PEA-BCC and FAST-BCC-FILTER algorithms (C_2, C_3), and the script used to generate the speedup data and graph plots (C_4).

Hardware: We used an NVIDIA L40 and NVIDIA L4 GPU for evaluation. The NVIDIA L40 features 18,176 CUDA cores with 48 GB of GDDR6 memory. The NVIDIA L4 features 7,424 CUDA cores with 24 GB of GDDR6 memory.

Using similar Ada Lovelace–based GPUs should result in performance comparable to that reported in the paper.

The minimum hardware requirement is a CUDA-capable GPU with compute capability 8.9 or higher and CUDA Toolkit 11.8 or newer. Lower versions may work but have not been tested.

Software: The computational artifact (Fast-BCC-Filter) is archived at:

https://doi.org/10.5281/zenodo.18669968

The source code is also publicly available on GitHub:

https://github.com/Abhijeetkumar96/FAST-BCC-on-GPUs

It can be obtained using:

```
$ git clone https://github.com/
Abhijeetkumar96/FAST-BCC-on-GPUs.git
```

Datasets / Inputs: The datasets used in our evaluation are hosted on OneDrive and can be accessed via:

https://tinyurl.com/fast-BCC-Filter-datasets

These files include the full-scale graph instances required to replicate the experimental results presented in Table II and Figures 3–7. After downloading, place the datasets in the `datasets/` directory in the project root as described in the README file. For quick validation and initial testing, a suite of smaller sample graphs is included directly within the GitHub repository and the Zenodo archive (A_1) under the `/datasets` directory.

Installation and Deployment: All implementations were compiled using NVCC v12.9 targeting compute capability 8.9 (`--sm_89`) and GCC v10.2.1. For plotting and post-processing results, we used Python 3.9.6 with `matplotlib`, and `gnuplot` v6.0. The build system relies on GNU Make v4.4.1. Lower or higher versions of the above tools may work but have not been tested.

Artifact Execution

Small sample datasets are available in the `datasets/` directory for quick functionality testing. The full-scale datasets used in the paper are hosted externally (see Section Datasets).

From the root directory of the repository, execute:

```
$ make
```

This command compiles all components of the artifact, including:

- CPU baseline implementations,
- GPU-based implementations (in-memory),
- External-memory (out-of-core) variants.

A. Running the Benchmark Suite

For convenience, we provide a unified script that executes the complete experimental workflow across all proposed methods and baseline implementations.

To run the full benchmarking suite:

```
$ export CPU_ROUNDS=5
$ export BATCH_SIZE=500000
$ ./run_all.sh
```

Environment Variables:

- `CPU_ROUNDS`: Number of repetitions for each CPU baseline execution (used to compute average runtime).
- `BATCH_SIZE`: Batch size parameter for the external-memory implementations.

Artifact Analysis (incl. Outputs)

The correctness of the artifact is evaluated by comparing the generated result figures with the corresponding figures presented in the paper. While absolute execution times will vary based on the specific CUDA device, the relative trends and speedup ratios should remain consistent.

Artifact Evaluation (AE)

Complete setup, execution, and analysis instructions are provided in the Artifact Description subsections above.